Systematische Datenanalyse mit Python

Modul: Coding und Digitalisierung

8. Oktober 2026

Einführung

Dieses Dokument beschreibt schrittweise, wie ein Datensatz in Python analysiert wird. Dabei wird zunächst die klassische Vorgehensweise mit Standard-Datentypen (Listen) und Schleifen gezeigt und anschließend der modernen, vektorbasierten Methode mit der Bibliothek `numpy` gegenübergestellt. Den Abschluss bildet die Visualisierung der Ergebnisse.

1 Datensatz erstellen

Wir beginnen mit der Erstellung eines einfachen Datensatzes in Form einer Liste. Dieser enthält normale Messwerte sowie bewusst eingefügte Ausreißer.

```
# Datensatz: Normale Werte liegen ca. zwischen 20 und 30
# Ausreisser sind die Werte 105 und 110
daten = [23, 25, 22, 26, 24, 23, 105, 25, 27, 24, 110, 26]
```

Listing 1: Datensatz mit Ausreißern erstellen

2 Analyse mit Standard-Python (Listen und Schleifen)

In reinem Python (ohne Zusatzbibliotheken) nutzen wir `for`-Schleifen, um die Ausreißer zu identifizieren, herauszufiltern und im Anschluss statistische Kennzahlen wie den Mittelwert und den Median zu berechnen.

```
# 1. Ausreisser suchen und filtern (Schwellenwert > 50)
bereinigte_daten = []
ausreisser = []

for wert in daten:
    if wert > 50:
        ausreisser.append(wert)
    else:
        bereinigte_daten.append(wert)

# 2. Mittelwert (Mean) berechnen
summe = 0
for wert in bereinigte_daten:
    summe += wert
mittelwert = summe / len(bereinigte_daten)

# 3. Median berechnen
```

```
sortierte_daten = sorted(bereinigte_daten)
n = len(sortierte_daten)
if n % 2 == 0:
    median = (sortierte_daten[n//2 - 1] + sortierte_daten[n//2]) / 2
else:
    median = sortierte_daten[n//2]

print(f"Klassisch - Mittelwert: {mittelwert}, Median: {median}")
```

Listing 2: Ausreißer filtern und Kennzahlen berechnen

3 Analyse mit NumPy

Die Bibliothek `numpy` optimiert diesen Prozess durch Arrays und Vektorisierung. Aufwendige `for`-Schleifen entfallen komplett.

```
import numpy as np

daten_np = np.array([23, 25, 22, 26, 24, 23, 105, 25, 27, 24, 110, 26])

# 1. Ausreisser filtern mit Boolescher Maskierung
bereinigte_daten_np = daten_np[daten_np <= 50]
ausreisser_np = daten_np[daten_np > 50]

# 2. Statistische Kennzahlen direkt abrufen
mittelwert_np = np.mean(bereinigte_daten_np)
median_np = np.median(bereinigte_daten_np)

print(f"NumPy - Mittelwert: {mittelwert_np}, Median: {median_np}")
```

Listing 3: Effiziente Analyse ohne Schleifen

4 Visualisierung der Daten

Um die Auswirkungen der Ausreißer sowie den bereinigten Zustand darzustellen, visualisieren wir die Daten mit `matplotlib`. Wir zeigen den Trend der Originaldaten als Liniendiagramm und den bereinigten Datensatz als Balkendiagramm.

```
import matplotlib.pyplot as plt

# Zwei Subplots nebeneinander erstellen
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# 1. Trend der Originaldaten (mit Ausreissern)
ax1.plot(daten_np, marker='o', color='red', linestyle='-')
ax1.set_title('Trend der Originalmesswerte')
ax1.set_xlabel('Index')
ax1.set_ylabel('Messwert')

# 2. Balkendiagramm der bereinigten Daten
indizes = range(len(bereinigte_daten_np))
ax2.bar(indizes, bereinigte_daten_np, color='blue')
ax2.set_title('Bereinigte Messwerte')
ax2.set_xlabel('Index')
ax2.set_ylabel('Messwert')
```

```
18
19 plt.tight_layout()
20 plt.show()
```

Listing 4: Trend und Balken mit Matplotlib